

Modelling

1. PRINCIPLES THAT GUIDE PRACTICE

1.1. Core Principles

Software engineering is guided by a collection of core principles that help in the application of a meaningful software process and the execution of effective software engineering methods. At the process level, core principles establish a philosophical foundation that guides a software team as it performs the framework and umbrella activities and produces a set of software engineering work products. At the practice level, core principles establish a collection of values and rules that can guide you in analyzing a problem, designing a solution, implementing and testing the solution, and ultimately deploying the software so stakeholders can use it.

1.1.1. *Principles That Guide Process*

In Part One of this book we discussed the importance of the software process and described several process models that have been proposed for software engineering work. Every project is unique, and every team is unique. That means you must adapt your process to best fit your needs. Regardless of the process model your team adopts, it contains elements of the generic process framework. The following set of core principles can be applied to this framework and to every software process. A simplified view of this framework is shown in Figure 1.1.

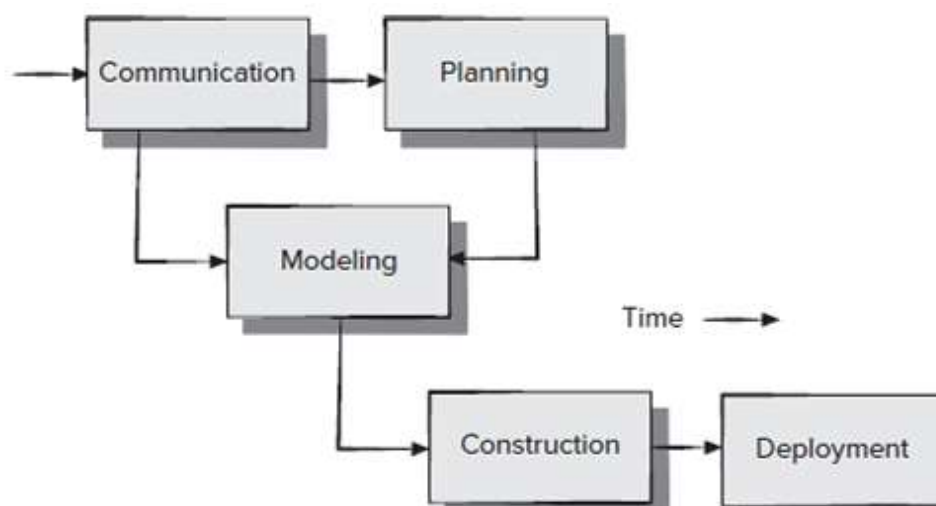


Figure 1.1 Simplified process framework

- **Principle 1. Be agile.** Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach. Every aspect of the work you do should emphasize economy of action—keep your technical approach as simple as possible,

keep the work products you produce as concise as possible, and make decisions locally whenever possible.

- **Principle 2. Focus on quality at every step.** The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.
- **Principle 3. Be ready to adapt.** Process is not a religious experience, and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.
- **Principle 4. Build an effective team.** Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.
- **Principle 5. Establish mechanisms for communication and coordination.** Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful end product. These are management issues, and they must be addressed.
- **Principle 6. Manage change.** The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved, and implemented.
- **Principle 7. Assess risk.** Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans. Some of these contingency plans will form the basis for security engineering tasks.
- **Principle 8. Create work products that provide value for others.** Create only those work products that provide value for other process activities, actions, or tasks. Every work product that is produced as part of software engineering practice will be passed on to someone else. Be sure that the work product imparts the necessary information without ambiguity or omission.

Part Four of this book focuses on project and process management issues and considers various aspects of each of these principles in some detail.

1.1.2. Principles That Guide Practice

Software engineering practice has a single overriding goal: to deliver on-time, high quality, operational software that contains functions and features that meet the needs of all stakeholders. To achieve this goal, you should adopt a set of core principles that guide your technical work. These principles have merit regardless of the analysis and design methods that you apply, the construction techniques (e.g., programming language, automated tools) that you use, or the verification and validation approach that you choose. The following set of core principles is fundamental to the practice of software engineering:

- **Principle 1. Divide and conquer.** Stated in a more technical manner, analysis and design should always emphasize separation of concerns (SoCs). A large problem is easier to solve if it is subdivided into a collection of elements (or concerns).
- **Principle 2. Understand the use of abstraction.** At its core, an abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase. When we use the abstraction spreadsheet, it is assumed that you understand what a spreadsheet is, the general structure of content that a spreadsheet presents, and the typical functions that can be applied to it. In software engineering practice, you use many different levels of abstraction, each imparting or implying meaning that must be communicated. In analysis and design work, a software team normally begins with models that represent high levels of abstraction (e.g., a spreadsheet) and slowly refines those models into lower levels of abstraction (e.g., a column or the SUM function).
- **Principle 3. Strive for consistency.** Whether it's creating an analysis model, developing a software design, generating source code, or creating test cases, the principle of consistency suggests that a familiar context makes software easier to use. As an example, consider the design of a user interface for a mobile app. Consistent placement of menu options, the use of a consistent color scheme, and the consistent use of recognizable icons help to create a highly effective user experience.
- **Principle 4. Focus on the transfer of information.** Software is about information transfer—from a database to an end user, from a legacy system to a WebApp, from an end user into a graphic user interface (GUI), from an operating system to an application, from one software component to another—the list is almost endless. In every case, information flows across an interface, and this means there are opportunities for errors, omissions, or ambiguity. The implication of this principle is that you must pay special attention to the analysis, design, construction, and testing of interfaces.
- **Principle 5. Build software that exhibits effective modularity.** Separation of concerns (Principle 1) establishes a philosophy for software. Modularity provides a mechanism for realizing the philosophy. Any complex system can be divided into modules (components), but good software engineering practice demands more. Modularity must be effective. That is, each module should focus exclusively on one well-constrained aspect of the system. Additionally, modules should be interconnected in a relatively simple manner to other modules, to data sources, and to other environmental aspects.
- **Principle 6. Look for patterns.** Software engineers use patterns as a means of cataloging and reusing solutions to problems they have encountered in the past. The use of these design patterns can be applied to wider systems engineering and systems integration problems, by allowing components in complex systems to evolve independently.
- **Principle 7. When possible, represent the problem and its solution from several different perspectives.** When a problem and its solution are examined from different

perspectives, it is more likely that greater insight will be achieved and that errors and omissions will be uncovered. The unified modeling language (UML) provides a means of describing a problem solution from multiple viewpoints.

- **Principle 8. Remember that someone will maintain the software.** Over the long term, software will be corrected as defects are uncovered, adapted as its environment changes, and enhanced as stakeholders request more capabilities. These maintenance activities can be facilitated if solid software engineering practice is applied throughout the software process.

These principles are not all you'll need to build high-quality software, but they do establish a foundation for every software engineering method discussed in this book.

1.2. Principles that guide each Framework Activity

In the sections that follow, we consider principles that have a strong bearing on the success of each generic framework activity defined as part of the software process. In many cases, the principles that are discussed for each of the framework activities are a refinement of the principles. They are simply core principles stated at a lower level of abstraction.

1.2.1. Communication Principles

Before customer requirements can be analyzed, modeled, or specified, they must be gathered through the communication activity. A customer has a problem that may be amenable to a computer-based solution. You respond to the customer's request for help. Communication has begun. But the road from communication to understanding is often full of potholes. Effective communication (among technical peers, with the customer and other stakeholders, and with project managers) is among the most challenging activities that you will confront. There are many ways to communicate, but it's important to recognize that not all are equal in richness or effectiveness (Figure 1.2). In this context, we discuss communication principles as they apply to customer communication. However, many of the principles apply equally to all forms of communication that occur within a software project.

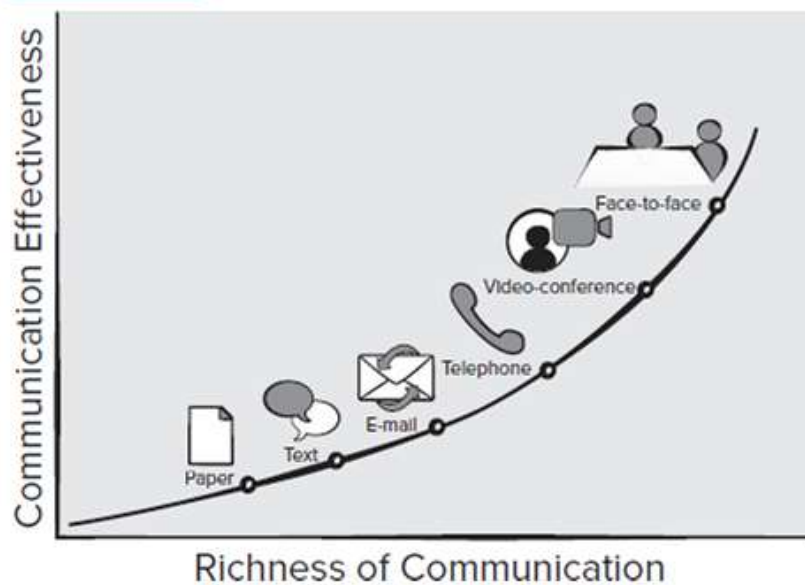


Figure1.2 Effectiveness of communication modes

- **Principle 1. Listen.** Before communicating, be sure you understand the point of view of the other party, know a bit about his or her needs, and then listen. Try to focus on the speaker's words, rather than formulating your response to those words. Ask for clarification if something is unclear, and avoid constant interruptions. Never become contentious in your words or actions (e.g., rolling your eyes or shaking your head) as a person is talking.
- **Principle 2. Prepare before you communicate.** Spend the time to understand the problem before you meet with others. If necessary, do some research to understand business domain jargon. If you have responsibility for conducting a meeting, prepare an agenda in advance of the meeting.
- **Principle 3. Someone should facilitate the activity.** Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction, (2) to mediate any conflict that does occur, and (3) to ensure that other principles are followed.
- **Principle 4. Face-to-face communication is best.** However, this communication usually works better when some other representation of the relevant information is present. For example, a participant may create a drawing or a "strawman" document that serves as a focus for discussion.
- **Principle 5. Take notes and document decisions.** Things have a way of falling into the cracks. Someone participating in the communication should serve as a "recorder" and write down all important points and decisions.
- **Principle 6. Strive for collaboration.** Collaboration and consensus occur when the collective knowledge of members of the team is used to describe product or system functions or features. Each small collaboration serves to build trust among team members and creates a common goal for the team.

- **Principle 7. Stay focused; modularize your discussion.** The more people are involved in any communication, the more likely that discussion will bounce from one topic to the next. The facilitator should keep the conversation modular, leaving one topic only after it has been resolved (however, see Principle 9)
- **Principle 8. If something is unclear, draw a picture.** Verbal communication goes only so far. A sketch or drawing can often provide clarity when words fail to do the job.
- **Principle 9. (a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified right now, move on.** Communication, like any software engineering activity, takes time. Rather than iterating endlessly, the people who participate should recognize that many topics require discussion (see Principle 2) and that “moving on” is sometimes the best way to achieve communication agility.
- **Principle 10. Negotiation is not a contest or a game.** It works best when both parties win. There are many instances in which you and other stakeholders must negotiate functions and features, priorities, and delivery dates. If the team has collaborated well, all parties have a common goal. Still, negotiation will demand compromise from all parties.

1.2.2. Planning Principles

The planning activity encompasses a set of management and technical practices that enable the software team to define a road map as it travels toward its strategic goal and tactical objectives. Try as we might, it's impossible to predict exactly how a software project will evolve. There is no easy way to determine what unforeseen technical problems will be encountered, what important information will remain undiscovered until late in the project, what misunderstandings will occur, or what business issues will change. And yet, a good software team must plan its approach. Often planning is iterative (Figure 1.3)

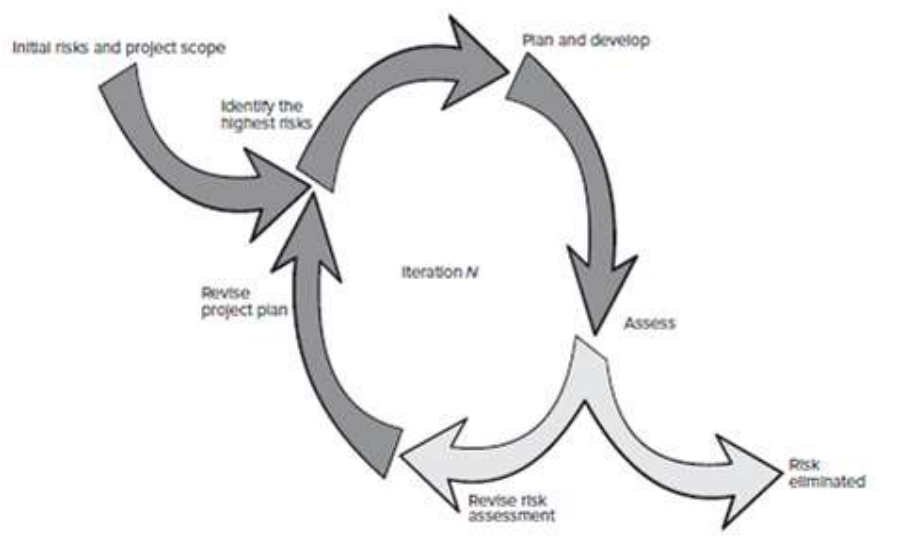


Figure1.3 Iterative planning

There are different planning philosophies.³ Some people are “minimalists,” arguing that change often obviates the need for a detailed plan. Others are “traditionalists,” arguing that the plan provides an effective road map and the more detail it has, the less likely the team will become lost.

What to do? On many projects, overplanning is time consuming and fruitless (too many things change), but underplanning is a recipe for chaos. Like most things in life, planning should be agile and conducted in moderation, enough to provide useful guidance for the team—no more, no less. Regardless of the rigor with which planning is conducted, the following principles always apply:

- **Principle 1. Understand the scope of the project.** It’s impossible to use a road map if you don’t know where you’re going. Scope provides the software team with a destination.
- **Principle 2. Involve stakeholders in the planning activity.** Stakeholders define priorities and establish project constraints. To accommodate these realities, software engineers must often negotiate order of delivery, time lines, and other project-related issues.
- **Principle 3. Recognize that planning is iterative.** A project plan is never engraved in stone. As work begins, it is very likely that things will change. The plan will need to be adjusted. Iterative, incremental process models include time for revising plans after the delivery of each software increment based on feedback received from users.
- **Principle 4. Estimate based on what you know.** The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team’s current understanding of the work to be done. If information is vague or unreliable, estimates will be equally unreliable.
- **Principle 5. Consider risk as you define the plan.** If you have identified risks that have high impact and high probability, contingency planning is necessary. The project plan (including the schedule) should be adjusted to accommodate the likelihood that one or more of these risks will occur.
- **Principle 6. Be realistic.** People don’t work 100 percent of every day. Changes will occur. Even the best software engineers make mistakes. These and other realities should be considered as a project plan is established.
- **Principle 7. Adjust granularity as you define the plan.** The term granularity refers to the detail with which some element of planning is represented or conducted. A high granularity plan provides significant work task detail that is planned over relatively short time increments (so that tracking and control occur frequently). A low-granularity plan provides broader work tasks that are planned over longer time periods. In general, granularity moves from high to low as the project time line moves away from the current date. Activities that won’t occur for many months do not require high granularity (too much can change).
- **Principle 8. Define how you intend to ensure quality.** The plan should identify how the software team intends to ensure quality. If technical reviews⁴ are to be conducted, they should be scheduled. If pair programming is to be used during construction, it should be explicitly defined within the plan.

- **Principle 9. Describe how you intend to accommodate change.** Uncontrolled change can obviate even the best planning. You should identify how changes are to be accommodated as software engineering work proceeds. For example, can the customer request a change at any time? If a change is requested, is the team obliged to implement it immediately? How is the impact and cost of the change assessed?
- **Principle 10. Track the plan frequently, and make adjustments as required.** Software projects fall behind schedule one day at a time. Therefore, it makes sense to track progress daily, looking for problem areas and situations in which scheduled work does not conform to actual work conducted. When slippage is encountered, the plan is adjusted accordingly.

To be most effective, everyone on the software team should participate in the planning activity. Only then will team members “sign up” to the plan.

1.2.3. Modeling Principles

We create models to gain a better understanding of the actual entity to be built. When the entity is a physical thing (e.g., a building, a plane, a machine), we can build a three dimensional (3D) model that is identical in form and shape but smaller in scale. However, when the entity to be built is software, our model must take a different form. It must be capable of representing the information that software transforms, the architecture and functions that enable the transformation to occur, the features that users desire, and the behavior of the system as the transformation is taking place. Models must accomplish these objectives at different levels of abstraction—first depicting the software from the customer’s viewpoint and later representing the software at a more technical level. Figure 1.4 shows how modeling may be used in agile software design.

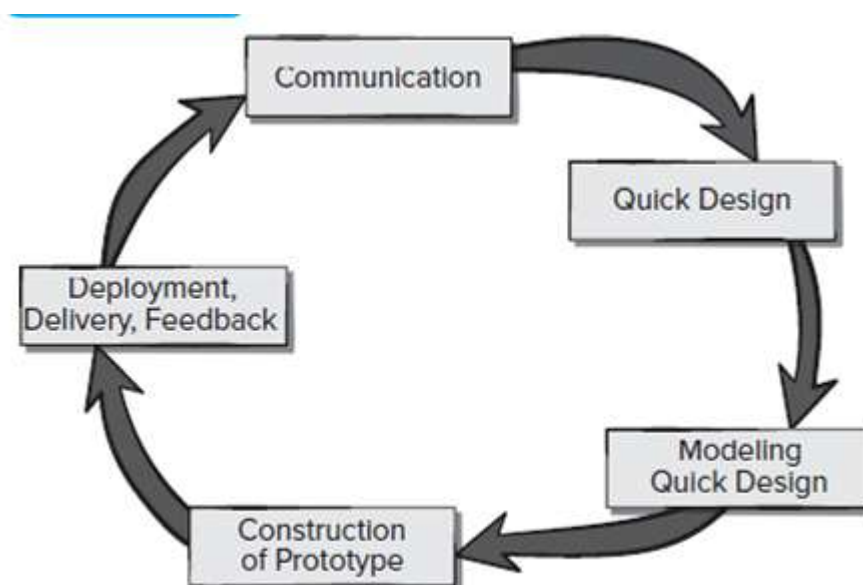


Figure1.4 Role of software modeling

- **Principle 1. The primary goal of the software team is to build software, not create models.** Agility means getting software to the customer in the fastest possible time. Models that make this happen are worth creating, but models that slow the process down or provide little new insight should be avoided.
- **Principle 2. Travel light—don't create more models than you need.** Every model that is created must be kept up to date as changes occur. More importantly, every new model takes time that might otherwise be spent on construction (coding and testing). Therefore, create only those models that make it easier and faster to construct the software.
- **Principle 3. Strive to produce the simplest model that will describe the problem or the software.** Don't overbuild the software [Amb02]. By keeping models simple, the resultant software will also be simple. The result is software that is easier to integrate, easier to test, and easier to maintain (to change). In addition, simple models are easier for members of the software team to understand and critique, resulting in an ongoing form of feedback that optimizes the end result.
- **Principle 4. Build models in a way that makes them amenable to change.** Assume that your models will change, but in making this assumption don't get sloppy. The problem with this attitude is that you may not create a reasonably complete requirements model, which means you'll create a design (design model) that will invariably miss important functions and features.
- **Principle 5. Be able to state an explicit purpose for each model that is created.** Every time you create a model, ask yourself why you're doing so. If you can't provide solid justification for the existence of the model, don't spend time on it.
- **Principle 6. Adapt the models you develop to the system at hand.** It may be necessary to adapt model notation or rules to the application; for example, a video game application might require a different modeling technique than real-time, embedded software for adaptive cruise control in an automobile.
- **Principle 7. Try to build useful models, but forget about building perfect models.** When building requirements and design models, a software engineer reaches a point of diminishing returns. That is, the effort required to make a model that is complete and internally consistent may not be worth the benefits of these properties. Iterating endlessly to make a model "perfect" does not serve the need for agility.
- **Principle 8. Don't become dogmatic about the syntax of the model.** If it communicates content successfully, representation is secondary. Although everyone on a software team should try to use consistent notation during modeling, the most important characteristic of the model is to communicate information that enables the next software engineering task. If a model does this successfully, incorrect syntax can be forgiven.
- **Principle 9. If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned.** If you are an experienced software

engineer, trust your instincts. Software work teaches many lessons—some of them on a subconscious level. If something tells you that a design model is doomed to fail (even though you can't prove it explicitly), you have reason to spend additional time examining the model or developing a different one.

- **Principle 10. Get feedback as soon as you can.** The intent of any model is to communicate information. It should stand on its own. Assume that you won't be there to explain the model. Every model should be reviewed by members of the software team. The intent of these reviews is to provide feedback that can be used to correct modeling mistakes, change misinterpretations, and add features or functions that were inadvertently omitted.

1.2.4. Construction Principles

The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user. In modern software engineering work, coding may be: (1) the direct creation of programming language source code, (2) the automatic generation of source code using an intermediate design like representation of the component to be built, or (3) the automatic generation of executable code using a fourth-generation programming language (e.g., Unreal4 Blueprints). The initial focus of testing is at the component level, often called unit testing. Other levels of testing include (1) integration testing (conducted as the system is constructed), (2) validation testing that assesses whether requirements have been met for the complete system (or software increment), and (3) acceptance testing that is conducted by the customer in an effort to exercise all required features and functions. Figure 1.5 shows where testing and test case design is placed in agile processes.

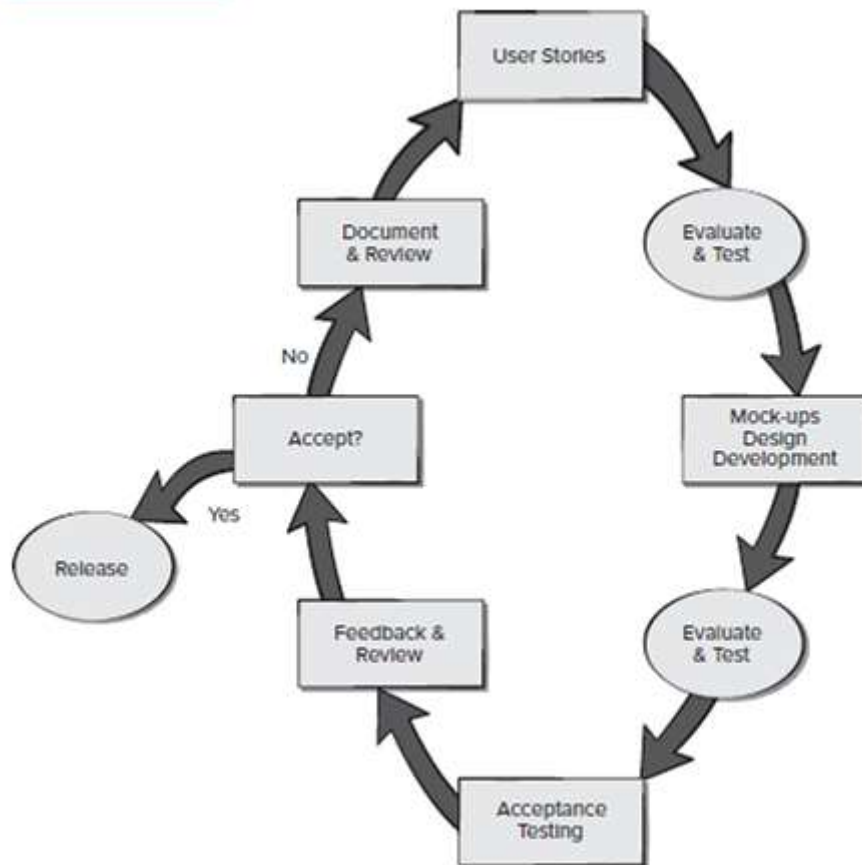


Figure1.5 Testing in agile processes

Testing is considered in detail in through. The following set of fundamental principles and concepts are applicable to coding and testing.

Coding Principles. The principles that guide the coding task are closely aligned with programming style, programming languages, and programming methods. There are some fundamental principles that might be followed:

Preparation Principles: Before you write one line of code, be sure you:

Principle 1. Understand the problem you're trying to solve.

Principle 2. Understand basic design principles and concepts.

Principle 3. Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.

Principle 4. Select a programming environment that provides tools that will make your work easier.

Principle 5. Create a set of unit tests that will be applied once the component you code is completed.

Coding Principles: As you begin writing code, be sure you:

Principle 6. Constrain your algorithms by following structured programming [Boh00] practice.

Principle 7. Consider the use of pair programming.

Principle 8. Select data structures that will meet the needs of the design.

Principle 9. Understand the software architecture and create interfaces that are consistent with it.

Validation Principles: After you've completed your first coding pass, be sure you:

Principle 10. Conduct a code walkthrough when appropriate.

Principle 11. Perform unit tests and correct errors you've uncovered.

Principle 12. Refactor the code to improve its quality.

Testing Principles. In a classic book on software testing, Glen Myers [Mye79] states a number of rules that can serve well as testing objectives:

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case is one that has a high probability of finding an as-yet-undiscovered error.
3. A successful test is one that uncovers an as-yet-undiscovered error.

These objectives imply a dramatic change in viewpoint for some software developers. They move counter to the commonly held view that a successful test is one in which no errors are found. Your objective is to design tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.

As a secondary benefit, testing demonstrates that software functions appear to be working according to specification and that behavioral and performance requirements appear to have been met. In addition, the data collected as testing is conducted provide a good indication of software reliability and some indication of software quality. Testing cannot show the absence of errors and defects; it can show only that software errors and defects are present (Figure 1.6). It is important to keep this (rather gloomy) statement in mind as testing is being conducted.



Figure1.6 Testing is never complete

- **Principle 1. All tests should be traceable to customer requirements.** The objective of software testing is to uncover errors. It follows that the most severe defects (from the customer's point of view) are those that cause the program to fail to meet its requirements.
- **Principle 2. Tests should be planned long before testing begins.** Test planning can begin as soon as the requirements model is complete. Detailed definition of test cases can begin as soon as the design model has been solidified. Therefore, all tests can be planned and designed before any code has been generated.
- **Principle 3. The Pareto principle applies to software testing.** In this context, the Pareto principle implies that 80 percent of all errors uncovered during testing will likely be traceable to 20 percent of all program components. The problem, of course, is to isolate these suspect components and to thoroughly test them.
- **Principle 4. Testing should begin "in the small" and progress toward testing "in the large."** The first tests planned and executed generally focus on individual components. As testing progresses, focus shifts to looking for errors in integrated clusters of components and ultimately in the entire system.
- **Principle 5. Exhaustive testing is not possible.** The number of path permutations for even a moderately sized program is exceptionally large. For this reason, it is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component level design have been exercised.

- **Principle 6. Apply to each module in the system a testing effort commensurate with its expected fault density.** These are often the newest modules or the ones that are least understood by the developers.
- **Principle 7. Static testing techniques can yield high results.** More than 85 percent of software defects originate in the software documentation (requirements, specifications, code walk-throughs, and user manuals). There may be value in testing the system documentation.
- **Principle 8. Track defects and look for patterns in defects uncovered by testing.** The total defects uncovered is a good indicator of software quality. The types of defects uncovered can be a good measure of software stability. Patterns of defects found over time can forecast numbers of expected defects.
- **Principle 9. Include test cases that demonstrate software is behaving correctly.** As software components are being maintained or adapted, unexpected interactions cause unintended side effects in other components. It is important to have a set of regression test cases ready to check system behavior after changes are made to a software product.

1.2.5. Deployment Principles

As we noted in Part One of this book, the deployment activity encompasses three actions: delivery, support, and feedback. Because modern software process models are evolutionary or incremental in nature, deployment happens not once, but several times as software moves toward completion. Each delivery cycle provides the customer and end users with an operational software increment that provides usable functions and features. Each support cycle provides documentation and human assistance for all functions and features introduced during all deployment cycles to date. Each feedback cycle provides the software team with important guidance that results in modifications to the functions, features, and approach taken for the next increment. Typical deployment actions are illustrated in Figure 1.7

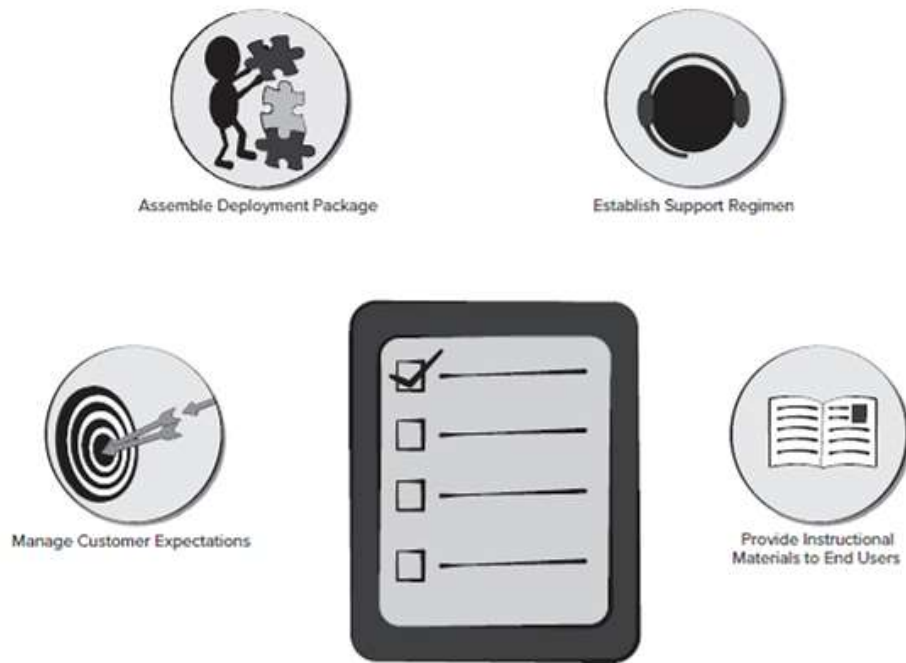


Figure1.7 Deployment actions

The delivery of a software increment represents an important milestone for any software project. Some key principles should be followed as the team prepares to deliver an increment:

- Principle 1. Customer expectations for the software must be managed.** Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately. This results in feedback that is not productive and ruins team morale. In her book on managing expectations, Naomi Karten [Kar94] states: “The starting point for managing expectations is to become more conscientious about what you communicate and how.” She suggests that a software engineer must be careful about sending the customer conflicting messages (e.g., promising more than you can reasonably deliver in the time period provided or delivering more than you promise for one software increment and then less than promised for the next).
- Principle 2. A complete delivery package should be assembled and tested.** All executable software, support data files, support documents, and other relevant information should be assembled and thoroughly beta-tested with actual users. All installation scripts and other operational features should be thoroughly exercised in all possible computing configurations (i.e., hardware, operating systems, peripheral devices, networking arrangements).
- Principle 3. A support regimen must be established before the software is delivered.** An end user expects responsiveness and accurate information when a question or problem arises. If support is ad hoc, or worse, nonexistent, the customer will become dissatisfied immediately. Support should be planned, support materials should be prepared, and appropriate record-keeping mechanisms should be established so that the software team can conduct a categorical assessment of the kinds of support requested.

- **Principle 4. Appropriate instructional materials must be provided to end users.** The software team delivers more than the software itself. Appropriate training aids (if required) should be developed; troubleshooting guidelines should be provided, and when necessary, a “what’s different about this software increment” description should be published.
- **Principle 5. Buggy software should be fixed first, delivered later.** Under time pressure, some software organizations deliver low-quality increments with a warning to the customer that bugs “will be fixed in the next release.” This is a mistake. There’s a saying in the software business: “Customers will forget you delivered a high-quality product a few days late, but they will never forget the problems that a low-quality product caused them. The software reminds them every day.”

2. Introduction to Software Modelling Techniques

2.1. Unified Modelling Language (UML)

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems .The UML is a language for
 - Visualizing
 - Specifying
 - Constructing
 - Documenting

Visualizing: The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously.

Specifying: The Specifying means building models that are precise, unambiguous, and complete. In particular, the UML addresses the specification of all the important analysis, design, and implementation decisions that must be made in developing and deploying a software-intensive system.

Constructing: The UML is not a visual programming language, but its models can be directly connected to a variety of programming languages. This means that it is possible to map from a model in the UML to a programming language such as Java, C++, or Visual Basic, or even to tables in a relational database or the persistent store of an object-oriented database. This mapping permits

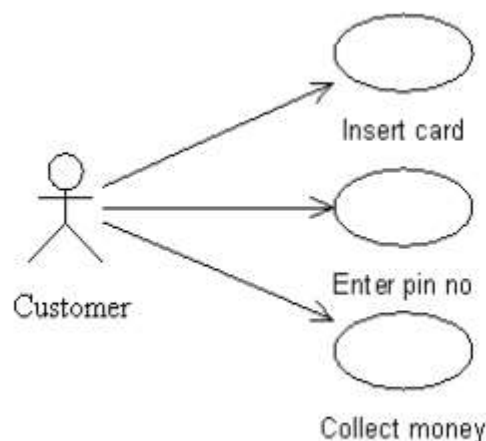
forward engineering: The generation of code from a UML model into a programming language. The reverse is also possible: You can reconstruct a model from an implementation back into the UML

Documenting: Documenting a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include

- Requirements
- Architecture
- Design
- Source code
- Project plans
- Tests
- Prototypes
- Releases

2.2. Use case Diagram

- A use case diagram shows a set of use cases and actors and their relationships.
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.



Interaction Diagrams

- Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams
- Interaction diagrams address the dynamic view of a system.

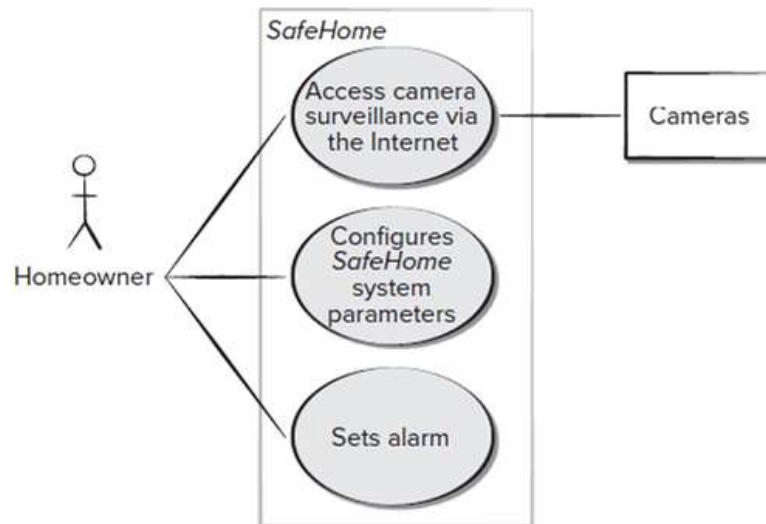


Figure 2.1 Preliminary use case diagram for the SafeHome system

The informal use cases are sometimes sufficient for requirements modeling. However, when a use case involves a critical activity or describes a complex set of steps with a significant number of exceptions, a more formal approach may be desirable.

The ACS-DCV use case shown in the sidebar follows a typical outline for formal use cases. The goal in context identifies the overall scope of the use case. The precondition describes what is known to be true before the use case is initiated. The trigger identifies the event or condition that “gets the use case started” [Coc01b]. The scenario lists the specific actions that are required by the actor and the appropriate system responses. Exceptions identify the situations uncovered as the preliminary use case is refined. Additional headings may or may not be included and are reasonably self-explanatory.

Most developers like to create graphical representation as they create use cases out of user stories. A diagrammatic representation can facilitate better understanding of the problem by all stakeholders, particularly when the scenario is complex. As we noted earlier in this book, UML provides use case diagramming capability. Figure 2.1 depicts a use case diagram for the SafeHome product. The use case diagram helps to show the relations among the use cases in the usage scenario. Each use case is represented by an oval. Only the ACS-DCV use case has been discussed in this section.\

Each modeling notation has limitations, and the UML use case is no exception. Like any other form of written description, a use case is only as good as its author(s). If the description is unclear, the use case can be misleading or ambiguous. A use case focuses on function and behavioral requirements and is generally inappropriate for nonfunctional requirements. For

situations in which the requirements model must have significant detail and precision (e.g., safety critical systems), a use case may not be sufficient.

However, scenario-based modeling is appropriate for a significant majority of all situations that you will encounter as a software engineer. If developed properly, the use case can provide substantial benefit as a modeling tool.

2.3. Class Diagram

- A Class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- A class consists of class name, attributes, operations and responsibilities.

As a first iteration at deriving a set of operations for an analysis class, you can again study a processing narrative (or use case) and select those operations that reasonably belong to the class. To accomplish this, the grammatical parse is again studied, and verbs are isolated. Some of these verbs will be legitimate operations and can be easily connected to a specific class. For example, from the SafeHome processing narrative presented earlier in this chapter, we see that “sensor is assigned a number and type” or “a master password is programmed for arming and disarming the system.” These phrases indicate several things: That an assign() operation is relevant for the Sensor class. That a program() operation will be applied to the System class. That arm() and disarm() are operations that apply to System class.

As a first iteration at deriving a set of operations for an analysis class, you can again study a processing narrative (or use case) and select those operations that reasonably belong to the class. To accomplish this, the grammatical parse is again studied, and verbs are isolated. Some of these verbs will be legitimate operations and can be easily connected to a specific class. For example, from the SafeHome processing narrative presented earlier in this chapter, we see that “sensor is assigned a number and type” or “a master password is programmed for arming and disarming the system.” These phrases indicate several things:

- That an assign() operation is relevant for the Sensor class.
- That a program() operation will be applied to the System class.
- That arm() and disarm() are operations that apply to System class.

Upon further investigation, it is likely that the operation program() will be divided into several more specific suboperations required to configure the system. For example, program() implies specifying phone numbers, configuring system characteristics (e.g., creating the sensor

table, entering alarm characteristics), and entering password(s). But for now, we specify program() as a single operation.

In addition to the grammatical parse, you can gain additional insight into other operations by considering the communication that occurs between objects. Objects communicate by passing messages to one another. Before continuing with the specification of operations, we explore this matter in a bit more detail.

In many instances, two analysis classes are related to one another in some fashion. In UML, these relationships are called associations. Referring to Figure 2.2, the FloorPlan class is defined by identifying a set of associations between FloorPlan and two other classes, Camera and Wall. The class Wall is associated with three classes that allow a wall to be constructed, WallSegment, Window, and Door.

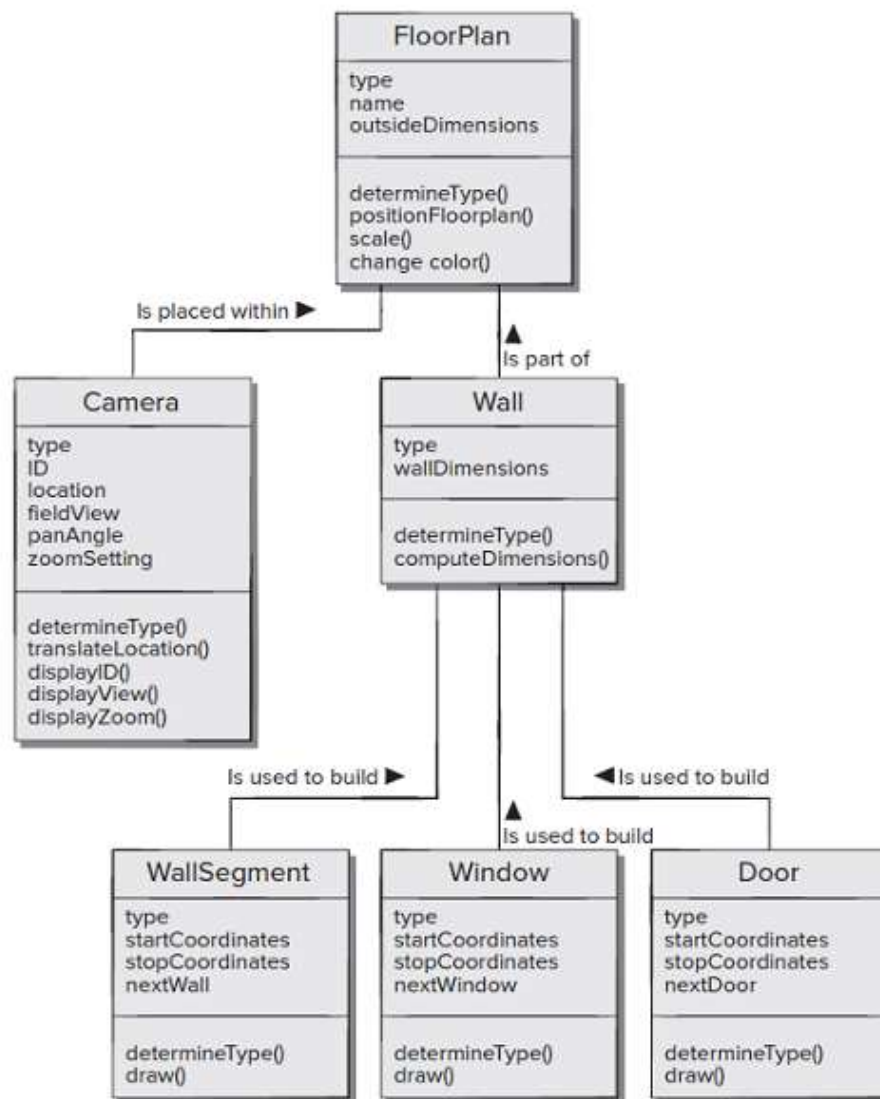


Figure 2.2 Class diagram for FloorPlan (see sidebar discussion)

2.4. Sequence Diagrams

- Sequence diagram emphasizes the time ordering of messages. It mainly shows set of objects and the messages send /receive by those objects which is concerned with time ordering of messages.
- It displays the time sequence of the objects participating in the interaction. This consists of the vertical dimension (time) and horizontal dimension (different objects).

The UML sequence diagram can be used for behavioral modeling. Sequence diagrams can also be used to show how events cause transitions from object to object. Once events have been identified by examining a use case, the modeler creates a sequence diagram—a representation of how events cause flow from one object to another as a function of time. The sequence diagram is a shorthand version of the use case. It represents key classes and the events that cause behavior to flow from class to class.

Figure 2.3 illustrates a partial sequence diagram for the SafeHome security function. Each of the arrows represents an event (derived from a use case) and indicates how the event channels behavior between SafeHome objects. Time is measured vertically (downward), and the narrow vertical rectangles represent time spent in processing an activity. States may be shown along a vertical time line.

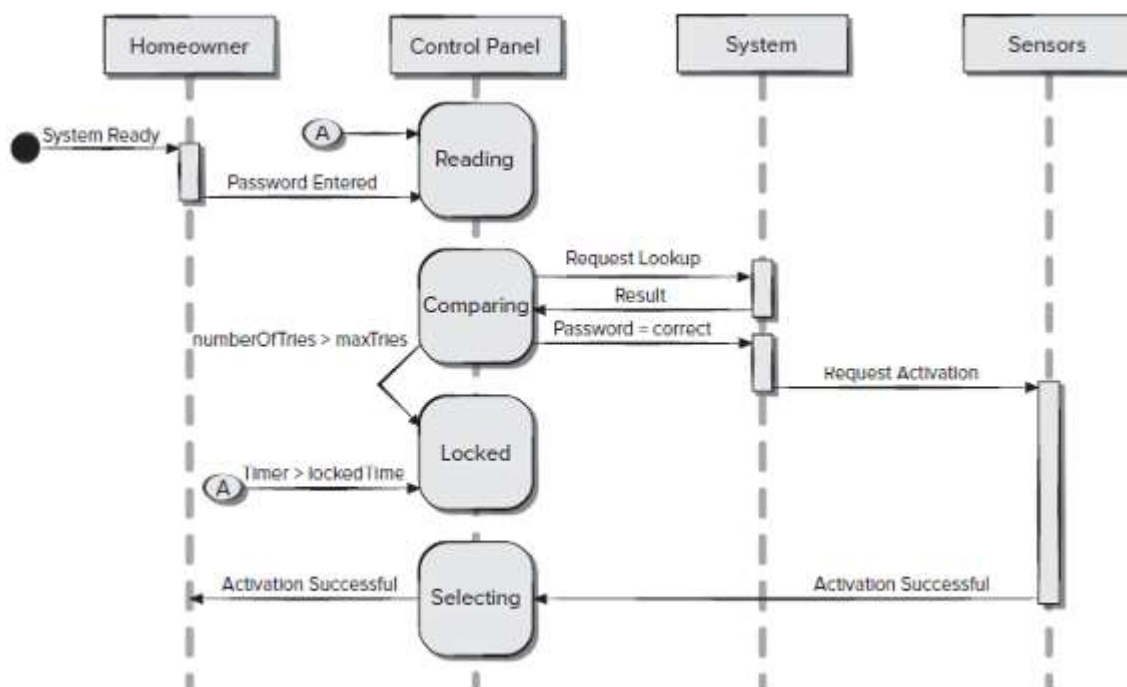


Figure 2.3 Sequence diagram (partial) for the SafeHome security function

2.5. Activity Diagrams

The UML activity diagram supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario. Many software engineers like to describe activity diagrams as a way of representing how a system reacts to internal events.

An activity diagram for the ACS-DCV use case is shown in Figure 2.4. It should be noted that the activity diagram adds additional detail not directly mentioned (but implied) by the use case. For example, a user may only attempt to enter userID and password a limited number of times. A decision diamond represents this below: “Prompt for reentry.”

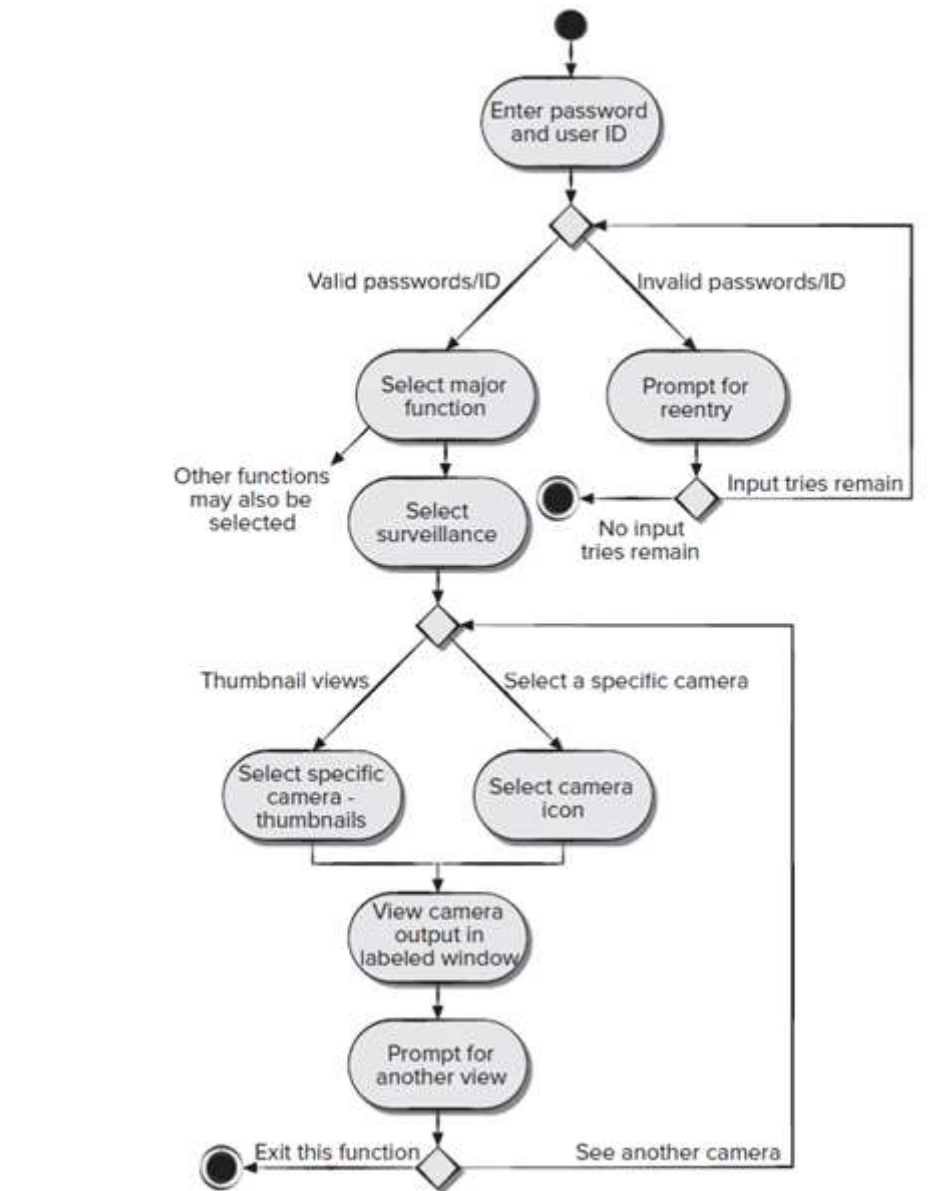


Figure 2.4 Activity diagram for Access camera surveillance via the Internet—display camera views function

2.6. Structural vs. Behavioural Modelling

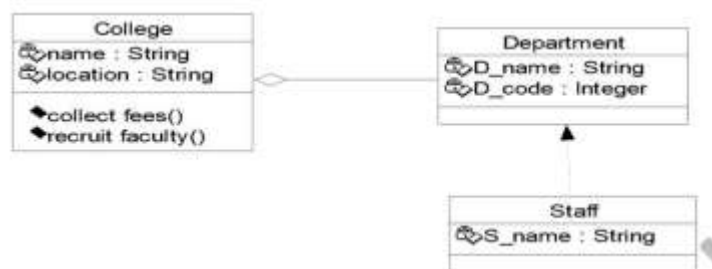
1. Structural Diagrams (static diagrams)
2. Behavioral Diagrams (Dynamic diagrams)

1. Structural Diagrams (static diagrams)

These are of 4 types

1. Class Diagram

- A Class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- A class consists of class name, attributes, operations and responsibilities.



2. Object diagram

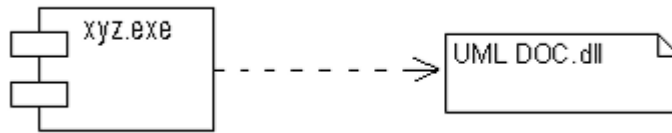
- Object diagrams represent static snapshots of instances of the things found in class diagrams.
- These diagrams address the static design view or static process view of a system.

An object diagram shows a set of objects and their relationships.

3. Component diagram

- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system.
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations

- It shows the organizations and dependencies among set of components. The static (view) implementation view of a system. It displays the high level packaged structure of the code itself.



4. Deployment diagram

- It shows the configuration of runtime processing nodes and the components that live on them it address the static deployment view of architecture.
- It displays the configuration of run-time processing elements and the software components, processes, and objects that live on them. Software component instances represent run-time manifestations of code units.

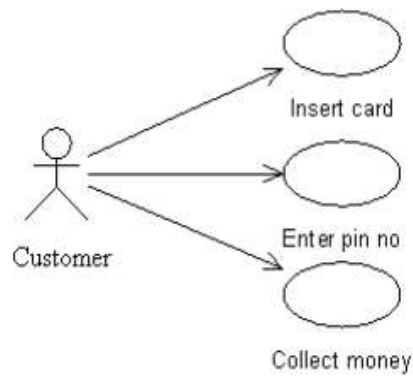


2. Behavioral Diagrams (Dynamic diagrams)

These are of 5 types

1. Use case diagram

- A use case diagram shows a set of use cases and actors and their relationships
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.

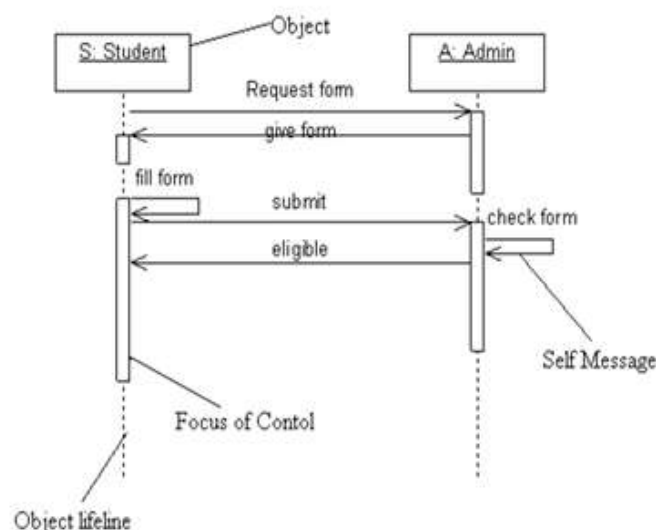


Interaction Diagrams

- Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams
- Interaction diagrams address the dynamic view of a system.

2. A sequence diagram

- Sequence diagram emphasizes the time ordering of messages. It mainly shows set of objects and the messages send /receive by those objects which is concerned with time ordering of messages.
- It displays the time sequence of the objects participating in the interaction. This consists of the vertical dimension (time) and horizontal dimension (different objects).



To show interaction between objects we use 3 types of messages.

Simple Messages: A Simple message shows how control is passed from one object to other without describing communication in detail i.e. without indicating whether it is synchronous or asynchronous message.

Synchronous Messages: If sender object waits for a reply from receiver object from destination, such messages are called Synchronous messages. Here, only one object can send a message at a given instance of time.

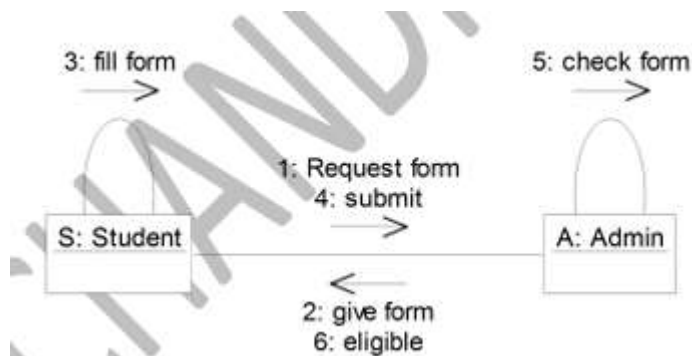
Asynchronous Messages: If sender object continues executing while target is processing the message then such messages are said to be Asynchronous messages. Here, multiple messages are executed at a time.

Object Lifeline: An Object life line is vertical dashed lines that represent the existence of an object over a period of time.

Focus of Control: It is represented by rectangle that shows the period of time during which an object performs some actions.

3. A collaboration diagram

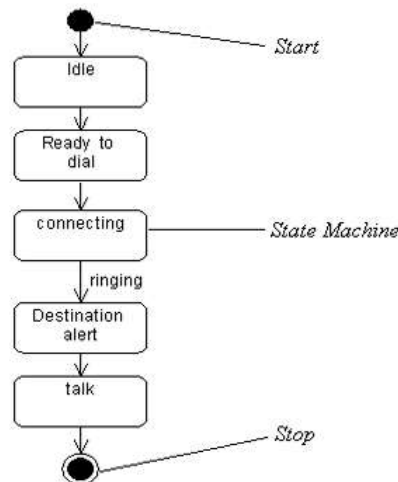
- A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages
- Sequence diagrams and collaboration diagrams are isomorphic, meaning that you can take one and transform it into the other.



4. Statechart diagram

- A statechart diagram shows a state machine, consisting of states, transitions, events, and activities.
- Statechart diagrams address the dynamic view of a system.

- They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object



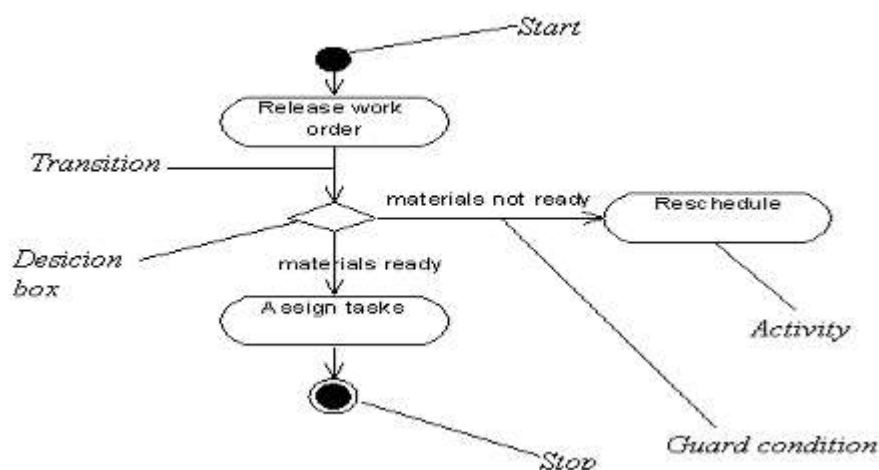
5. Activity diagram

- An activity diagram is a special kind of a statechart diagram that shows the flow from activity to activity within a system
- Activity diagrams address the dynamic view of a system
- They are especially important in modeling the function of a system and emphasize the flow of control among objects.

Activity: It is a major task that must take place in order to fulfill an operation contract.

Initial Activity: This shows the starting point of the flow. It is denoted by solid circle

Final Activity: This shows the end of the flow in the activity diagram. It is denoted by a solid circle nested in a circle.



Decision Box: A point in an Activity diagram where a flow splits into several mutually exclusive guarded flows. It has one incoming transition and two outgoing transitions.

Forking and Joining: We use synchronization bar to specify the forking and joining of parallel flows of control.

A synchronization bar is a thick horizontal or vertical line.

A Fork may have one incoming transition and two or more outgoing transitions, each of which represents an independent flow of control.

A Join may have two or more incoming transitions and one outgoing transition.

3. DESIGN CONCEPTS

Several fundamental software design concepts have evolved over the history of software engineering. Although the degree of interest in these concepts has varied over the years, each has stood the test of time. Each provides the software designer with a foundation from which more sophisticated design methods can be applied. Each helps you define criteria that can be used to partition software into individual components, separate out data structure detail from a conceptual representation of the software, and establish uniform criteria that define the technical quality of a software design. These concepts help developers design the software actually needed, rather than simply focusing on creating any old working program.

A set of fundamental software design concepts has evolved over the history of software engineering. Although the degree of interest in each concept has varied over the years, each has stood the test of time. Each provides the software designer with a foundation from which more sophisticated design methods can be applied. Each helps you answer the following questions:

- What criteria can be used to partition software into individual components?
- How is function or data structure detail separated from a conceptual representation of the software?
- What uniform criteria define the technical quality of a software design?

M. A. Jackson [Jac75] once said: “The beginning of wisdom for a [software engineer] is to recognize the difference between getting a program to work, and getting it right.” Fundamental software design concepts provide the necessary framework for “getting it right.”

3.1. Abstraction

When you consider a modular solution to any problem, many levels of abstraction can be posed. At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment (e.g., a user story). At lower levels of abstraction, a more detailed description of the solution is provided. Problem-oriented terminology is coupled with implementation-oriented terminology to state a solution (e.g., use case). Finally, at the lowest level of abstraction, the solution is stated in a manner that can be directly implemented (e.g., pseudocode).

As different levels of abstraction are developed, you work to create both procedural and data abstractions. A procedural abstraction refers to a sequence of instructions that have a specific and limited function. The name of a procedural abstraction implies these functions, but specific details are suppressed. An example of a procedural abstraction would be the word use for a camera in the SafeHome system. Use implies a long sequence of procedural steps (e.g., activate the SafeHome system on a mobile device, log on to the SafeHome system, select a camera to preview, locate the camera controls on mobile app user interface, etc.).

A data abstraction is a named collection of data that describes a data object. In the context of the procedural abstraction open, we can define a data abstraction called camera. Like any data object, the data abstraction for camera would encompass a set of attributes that describe the camera (e.g., camera ID, location, field view, pan angle, zoom). It follows that the procedural abstraction use would make use of information contained in the attributes of the data abstraction camera.

3.2. Modularity

Modularity is the most common manifestation of separation of concerns. Software is divided into separately named and addressable components, sometimes called modules, that are integrated to satisfy problem requirements.

It has been stated that “modularity is the single attribute of software that allows a program to be intellectually manageable” [Mye78]. Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer. The number of control paths, span

of reference, number of variables, and overall complexity would make understanding close to impossible. In almost all instances, you should break the design into many modules, hoping to make understanding easier and reduce the cost required to build the software.

Recalling our discussion of separation of concerns, it is possible to conclude that if you subdivide software indefinitely the effort required to develop it will become negligibly small! Unfortunately, other forces come into play, causing this conclusion to be (sadly) invalid. Referring to Figure 3.1, the effort (cost) to develop an individual software module tends to decrease as the total number of modules increases.

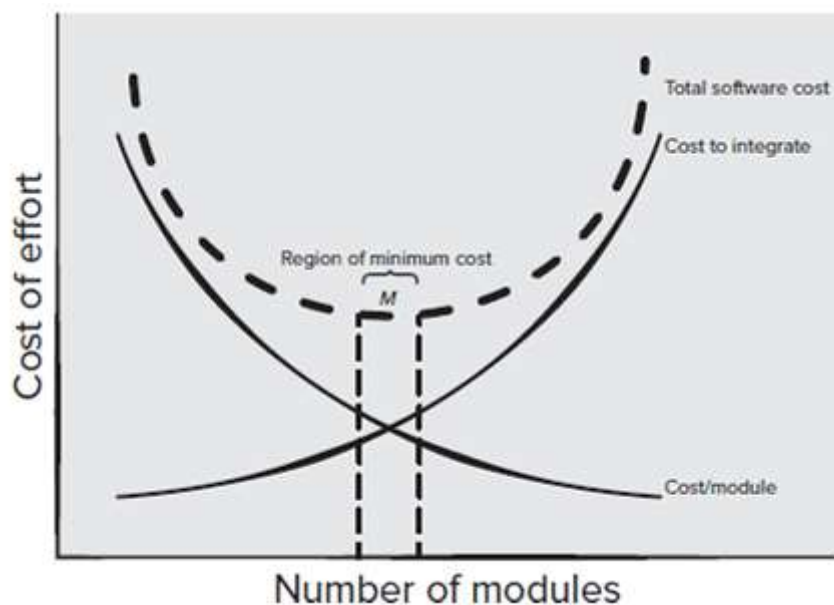


Figure 3.1 Modularity and software cost

Given the same set of requirements, the more modules used in your program means smaller individual sizes. However, as the number of modules grows, the effort (cost) associated with integrating modules with each other grows. These characteristics lead to a total cost or effort curve, shown in Figure 3.1. There is a number, M , of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict M with assurance.

The curves shown in Figure 3.1 do provide useful qualitative guidance when modularity is considered. You should modularize, but care should be taken to stay in the vicinity of M . Using too few modules or too many modules should be avoided. But how do you know the vicinity of M ? How modular should you make software? The answers to these questions require an understanding of other design concepts.

You modularize a design (and the resulting program) so that development can be more easily planned, software increments can be defined and delivered, changes can be more easily accommodated, testing and debugging can be conducted more efficiently, and long-term maintenance can be conducted without serious side effects.

3.3. Functional Independence

The concept of functional independence is a direct outgrowth of separation of concerns, modularity, and the concepts of abstraction and information hiding. In landmark papers on software design, Wirth [Wir71] and Parnas [Par72] each allude to refinement techniques that enhance module independence. Later work by Stevens, Myers, and Constantine [Ste74] solidified the concept.

Functional independence is achieved by developing modules with “single-minded” function and an “aversion” to excessive interaction with other modules. Stated another way, you should design software so that each module addresses a specific subset of requirements and has a simple interface when viewed from other parts of the program structure.

It is fair to ask why independence is important. Software with effective modularity, that is, independent modules, is easier to develop because function can be compartmentalized and interfaces are simplified (consider the ramifications when development is conducted by a team). Independent modules are easier to maintain (and test) because secondary effects caused by design or code modification are limited, error propagation is reduced, and reusable modules are possible. To summarize, functional independence is a key to good design, and design is the key to software quality. Evaluation of your CRC card model can help you spot problems with functional independence. User stories that contain many instances of words such as and or except are not likely to encourage you to design modules that are “single-minded” system functions.

Independence is assessed using two qualitative criteria: cohesion and coupling. Cohesion is an indication of the relative functional strength of a module. Coupling is an indication of the relative interdependence among modules.

Cohesion is a natural extension of the information-hiding concept described in Sec A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing. Although you should always strive for high cohesion (i.e., single-mindedness), it is often necessary and advisable to have a software component perform multiple functions. However,

“schizophrenic” components (modules that perform many unrelated functions) are to be avoided if a good design is to be achieved.

Coupling is an indication of interconnections among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface. In software design, you should strive for the lowest possible coupling. Simple connectivity among modules results in software that is easier to understand and less likely to propagate errors found in one module to other system modules.

3.4. Information Hiding

The concept of modularity leads you to a fundamental question: “How do I decompose a software solution to obtain the best set of modules?” The principle of information hiding [Par72] suggests that modules should be “characterized by design decisions that (each) hides from all others.” In other words, modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information.

Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function. Abstraction helps to define the procedural (or informational) entities that make up the software. Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module [Ros75].

The use of information hiding as a design criterion for modular systems provides the greatest benefits when modifications are required during testing and later during software maintenance. Because most data and procedural detail are hidden from other parts of the software, inadvertent errors introduced during modification are less likely to propagate to other locations within the software.